

OpenBao Security Evolution Analysis Report

Versions 2.4.0 to 2.7.0

Adrien SALES, geol, Trivy and Claude Code

September 24, 2026

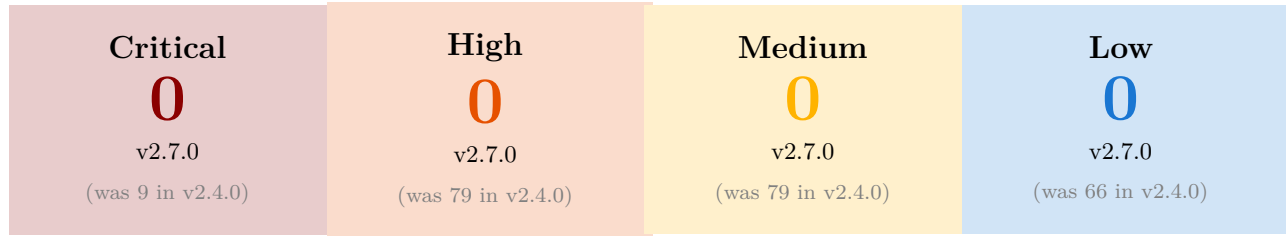
Abstract

This document provides a comprehensive analysis of the security evolution in OpenBao, covering major and minor releases from version 2.4.0 to 2.7.0.

Utilizing the Trivy vulnerability scanner on the respective OCI images, this report highlights the continuous improvement in the security posture of the project. To ensure a methodologically sound comparison, **all fifteen image versions were rescanned in a single session against an identical Trivy vulnerability database**, eliminating the database-drift bias that arises when versions are scanned weeks apart. The analysis encompasses vulnerability trends by severity and provides insights into the impact of base image updates (Alpine Linux) on the overall risk profile. The findings demonstrate a significant and consistent reduction in critical and high-severity vulnerabilities, validating the project's commitment to security and the importance of adopting the latest stable releases.

1 Executive Summary

1.1 Key Security Metrics at a Glance



1.2 Key Takeaways

- **Lowest Vulnerability Count:** v2.7.0 — 1 total finding, 0 Critical/High/Medium/Low, Security Score: **100.0/100** (a perfect weighted score: the single remaining item is Unknown-severity and therefore carries no weight)
- **Highest Vulnerability Count:** v2.4.0/v2.4.1 — 235 total vulnerabilities, 9 Critical, Security Score: **29.1/100**
- **Biggest Improvement:** v2.6.3 → v2.7.0 — **88.9% reduction** (9 → 1), as the 2.7.0 release closes every outstanding OpenBao core advisory at once
- **Recommended Minimum:** v2.6.3+ for production use (Score ≥ 97.0/100). Because this rescan used a Trivy database four months newer than the previous edition of this report, **every** version's counts rose against their previously published values (e.g. v2.6.1: 10 → 47, v2.5.5: 15 → 55) — a textbook illustration of CVE database drift, and the reason all fifteen tags are always rescanned together.
- **Overall Improvement:** **99.6% reduction** from v2.4.0 to v2.7.0 (235 → 1 vulnerabilities)
- **Latest Version:** v2.7.0 — The first release in this series with **no Critical, High, Medium or Low findings at all**. It clears the seven long-standing OpenBao core CVEs (CVE-2024-8185, CVE-2024-9180, CVE-2025-59043, CVE-2025-64761, CVE-2026-45808, CVE-2026-46358, CVE-2026-46405) that had persisted in all fourteen prior releases, and inherits the Alpine 3.24.2 base introduced in v2.6.3 that zeroes the OS layer.
- **Vulnerability Remediation:** 97–99% of CVEs are fixable across v2.4.0–v2.6.2. v2.6.3 drops to 89% and v2.7.0 to 0%, but this is a sign of health, not risk: as the fixable backlog is cleared, the only finding left is GO-2026-5932, an advisory on the unmaintained golang.org/x/crypto/openpgp package for which no upstream fix exists.
- **Versioning Policy:** OpenBao follows Semantic Versioning. Upgrading from v2.5.x to v2.5.y is **strongly encouraged** and considered low-risk. These updates are essential to benefit from security patches, bug fixes, and performance improvements without breaking compatibility. Staying on the latest patch of a minor release is the recommended security best practice.

1.3 Security Posture Scores

The Security Posture Score weights each finding by severity (Critical: 10 points, High: 5, Medium: 2, Low: 1) and expresses the result against a **fixed reference ceiling of 1000 weighted points**:

$$\text{Score} = 100 - \frac{\text{weighted points}}{1000} \times 100$$

The choice of a fixed ceiling is deliberate. Earlier editions of this report normalized against the *worst version in the corpus*, which had three defects: the worst version always scored exactly 0 by construction

rather than by measurement, every score silently re-scaled whenever a version was added or the CVE database moved, and scores could not be compared against any other product's report. With a constant denominator, a score means the same thing in every edition and for every product, and no version is pinned to either endpoint — v2.4.0 now scores 29.1 rather than a definitionally-zero 0.0, which is the more truthful statement that it is bad but not maximally bad.

Table 1: Security Posture Score by Version

Version	Critical	High	Medium	Low	Total	Weighted	Score /100
2.4.0	9	79	79	66	233	709	29.1
2.4.1	9	79	79	66	233	709	29.1
2.4.3	9	78	62	63	212	667	33.3
2.4.4	9	78	60	63	210	663	33.7
2.5.0	9	71	48	44	172	585	41.5
2.5.1	8	70	48	44	170	570	43.0
2.5.2	6	69	47	42	164	541	45.9
2.5.3	1	45	36	37	119	344	65.6
2.5.4	1	39	33	37	110	308	69.2
2.5.5	1	23	15	15	54	170	83.0
2.6.0	0	22	13	13	48	149	85.1
2.6.1	0	20	13	13	46	139	86.1
2.6.2	0	11	12	13	36	92	90.8
2.6.3	0	5	2	1	8	30	97.0
2.7.0	0	0	0	0	0	0	100.0

Note on comparability with earlier editions: scores in this edition differ from previously published figures for *two* independent reasons — the scoring denominator changed (described above), and the CVE database is four months newer (Section 8). Neither reflects a change to any image. Scores within this report are mutually comparable; scores quoted from an earlier edition are not.

1.4 Risk Level Matrix

This matrix provides a quick visual assessment of the risk level associated with each version based on vulnerability composition.

Table 2: Risk Level Assessment by Version

Version	Critical	High	Medium	Low	Risk Level
2.4.0	9	79	79	66	HIGH
2.4.1	9	79	79	66	HIGH
2.4.3	9	78	62	63	HIGH
2.4.4	9	78	60	63	HIGH
2.5.0	9	71	48	44	HIGH
2.5.1	8	70	48	44	HIGH
2.5.2	6	69	47	42	MEDIUM
2.5.3	1	45	36	37	MEDIUM
2.5.4	1	39	33	37	MEDIUM
2.5.5	1	23	15	15	LOW
2.6.0	0	22	13	13	LOW
2.6.1	0	20	13	13	LOW
2.6.2	0	11	12	13	LOW
2.6.3	0	5	2	1	LOW
2.7.0	0	0	0	0	LOW

Risk Level Guidelines:

- **HIGH** (Score < 45, i.e. more than 550 weighted points): Not recommended for production. Contains critical vulnerabilities requiring immediate attention.
- **MEDIUM** (Score 45–75, i.e. 250–550 weighted points): Use with caution. Suitable for development environments with proper monitoring.
- **LOW** (Score > 75, i.e. fewer than 250 weighted points): Recommended for production. Minimal security concerns with available patches.

2 Introduction

This report provides an in-depth analysis of the security evolution in OpenBao from version 2.4.0 through 2.7.0. The assessment was performed using the Trivy vulnerability scanner on the official OCI images for each release. The primary objective is to track the evolution of the project's security posture over time, identifying trends in vulnerability counts across different severity levels and evaluating the impact of maintenance and updates, such as base operating system upgrades.

2.1 OpenBao Versioning Policy and Release Strategy

OpenBao adheres to **Semantic Versioning** principles, ensuring predictable and transparent release management. The project follows a versioning scheme similar to industry standards, where version numbers follow the pattern `MAJOR.MINOR.PATCH`.

Similar to PostgreSQL's versioning policy¹, OpenBao's minor releases are designed to be **low-risk updates** that focus on:

- Fixes for frequently-encountered bugs
- Low-risk stability improvements
- Security vulnerability patches
- Data corruption prevention

The OpenBao community considers performing minor upgrades to be less risky than continuing to run an old minor version. This philosophy is validated by the security analysis presented in this report, where consistent minor version updates demonstrate significant vulnerability reduction without introducing breaking changes.

As PostgreSQL documentation states: *“Minor releases only contain fixes for frequently-encountered bugs, low-risk fixes, security issues, and data corruption problems. The community considers performing minor upgrades to be less risky than continuing to run an old minor version.”* This same principle applies to OpenBao's release strategy, making the adoption of latest minor releases a recommended security best practice.

3 Tooling Details

This comparative analysis report was generated and compiled using several tools, including an AI agent (Claude Code), to assess and present the vulnerability data.

3.1 Claude Code

This entire report, including the analysis, data extraction, and LaTeX document generation, was orchestrated by an instance of Claude Code. The agent was responsible for:

1. Interacting with vulnerability scanning tools.
2. Parsing their outputs.
3. Performing comparative data analysis.
4. Constructing the LaTeX document structure.
5. Generating the textual content and charts.
6. Compiling the $\text{\LaTeX}$ document into a PDF.
7. Obtaining End-of-Life (EOL) information for OpenBao versions from Geol.

¹PostgreSQL Versioning Policy: <https://www.postgresql.org/support/versioning/>

3.2 Geol

Geol is a tool used for providing product lifecycle and support information.

- **Version:** 2.21.4
- **Usage:** Querying OpenBao release lifecycles and support status.

3.3 Trivy

Trivy (v0.74.0) is a comprehensive security scanner for container images. It was the primary tool for vulnerability assessment in this report. **All fifteen image tags were rescanned on the same day against a single, shared vulnerability database** (downloaded once with `--download-db-only`, then reused via `--skip-db-update`), so that every version is evaluated against the exact same set of known CVEs.

- **Vulnerability Database Updated:** September 24, 2026
- **Scanning Commands Examples:**

```
trivy image --format json --output openbao_2.4.0.json openbao/openbao:2.4.0
trivy image --format json --output openbao_2.7.0.json openbao/openbao:2.7.0
```

- **Analyzed Image Tags:** 2.4.0, 2.4.1, 2.4.3, 2.4.4, 2.5.0, 2.5.1, 2.5.2, 2.5.3, 2.5.4, 2.5.5, 2.6.0, 2.6.1, 2.6.2, 2.6.3, 2.7.0.

4 Tool Homepages

- **Claude Code:** <https://claude.com/claude-code>
- **Geol:** <https://github.com/opt-nc/geol>
- **Trivy:** <https://trivy.dev/>

5 Base Image Analysis

The security posture of an OCI image is heavily influenced by its base operating system. OpenBao utilizes Alpine Linux, a security-oriented, lightweight Linux distribution based on musl libc and busybox.

5.1 Base OS Freshness

Using the newest Alpine patch available is as important as the choice of major/minor branch: Alpine ships synchronized security-patch releases across all supported branches roughly every 4–10 weeks. For each OpenBao release, we cross-referenced **geol** (Alpine Linux cycle/EOL data) with the exact **Docker Hub tag publish timestamps** for every `alpine:3.2x.y` tag to reconstruct which Alpine patch was actually the latest one publicly available on the day each OpenBao image was built — not merely the latest one known today (the two can differ, since new Alpine patches are released continuously).

Table 3: Base OS Versions (Alpine Linux) and Freshness at Release Time

OpenBao Version	Base OS	Version Used	Latest Alpine Available (that day)	Latest at Release?
2.4.0	Alpine	3.22.1	3.22.1	Yes
2.4.1	Alpine	3.22.1	3.22.1	Yes
2.4.3	Alpine	3.22.2	3.22.2	Yes
2.4.4	Alpine	3.22.2	3.22.2	Yes
2.5.0	Alpine	3.23.3	3.23.3	Yes
2.5.1	Alpine	3.23.3	3.23.3	Yes
2.5.2	Alpine	3.23.3	3.23.3	Yes
2.5.3	Alpine	3.23.4	3.23.4	Yes
2.5.4	Alpine	3.23.4	3.23.4	Yes
2.5.5	Alpine	3.24.1	3.24.1	Yes
2.6.0	Alpine	3.24.1	3.24.1	Yes
2.6.1	Alpine	3.24.1	3.24.1	Yes
2.6.2	Alpine	3.24.1	3.24.1	Yes
2.6.3	Alpine	3.24.2	3.24.2	Yes
2.7.0	Alpine	3.24.2	3.24.2	Yes

Finding: every single analyzed OpenBao release — all fifteen — shipped with the Alpine patch that was the most recent one publicly available on its build date (e.g. v2.5.3 was built 2026-04-20, five days after Alpine 3.23.4 published on 2026-04-15; v2.6.3 and v2.7.0 were both built 2026-09-23, five days after Alpine 3.24.2 published on 2026-09-18). None of the “stale-looking” minor-version gaps in Table 3 (e.g. v2.6.0, v2.6.1 and v2.6.2 all staying on Alpine 3.24.1) reflect a missed update: in each case, no newer Alpine patch had been published yet at the time of that OpenBao build. This indicates disciplined, continuous base-image rebasing by the OpenBao maintainers rather than periodic/lagging updates.

6 Security Evolution Analysis (v2.4.0 to v2.7.0)

6.1 Vulnerability Evolution Table

Table 4: Vulnerability Counts: v2.4.0 through v2.7.0

Version	Critical	High	Medium	Low	Unknown
2.4.0	9	79	79	66	2
2.4.1	9	79	79	66	2
2.4.3	9	78	62	63	2
2.4.4	9	78	60	63	2
2.5.0	9	71	48	44	2
2.5.1	8	70	48	44	2
2.5.2	6	69	47	42	2
2.5.3	1	45	36	37	2
2.5.4	1	39	33	37	2
2.5.5	1	23	15	15	1
2.6.0	0	22	13	13	1
2.6.1	0	20	13	13	1
2.6.2	0	11	12	13	1
2.6.3	0	5	2	1	1
2.7.0	0	0	0	0	1

6.2 Overall Security Trend Chart

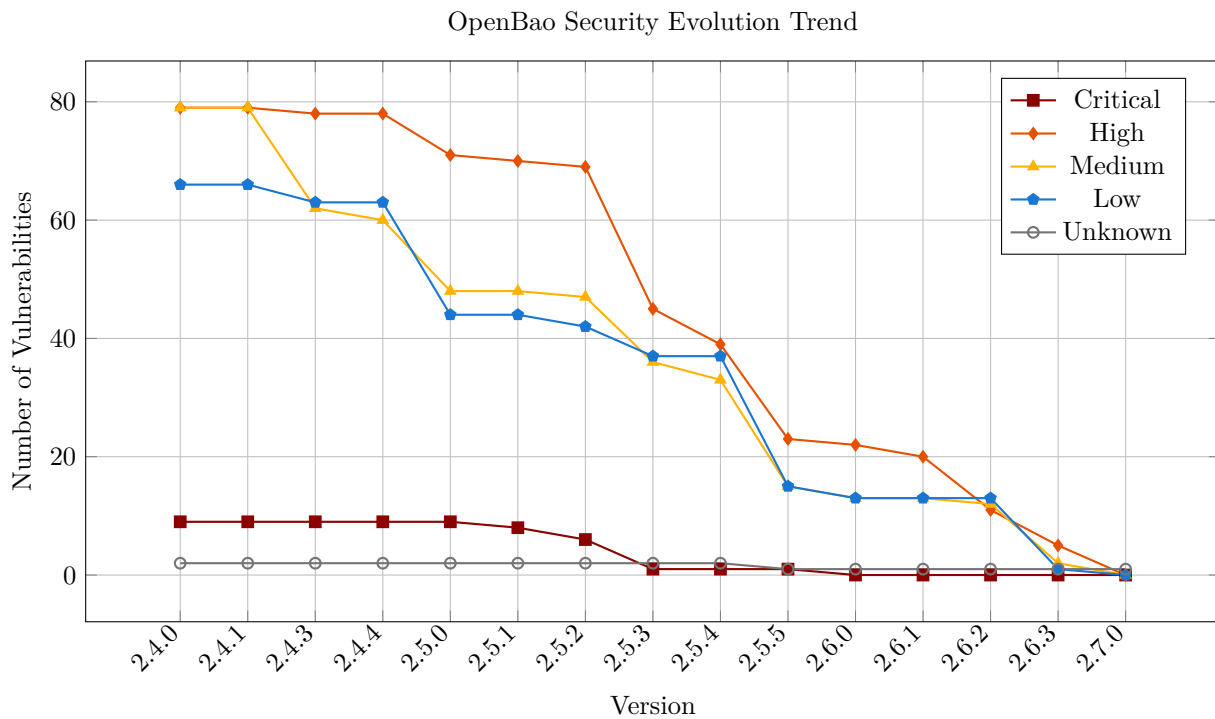


Figure 1: Security Improvement Trend: v2.4.0 to v2.7.0

6.3 Stacked Area Chart: Total Vulnerability Composition

This stacked area chart provides a complementary view of the security evolution, showing the total volume and composition of vulnerabilities by severity level.

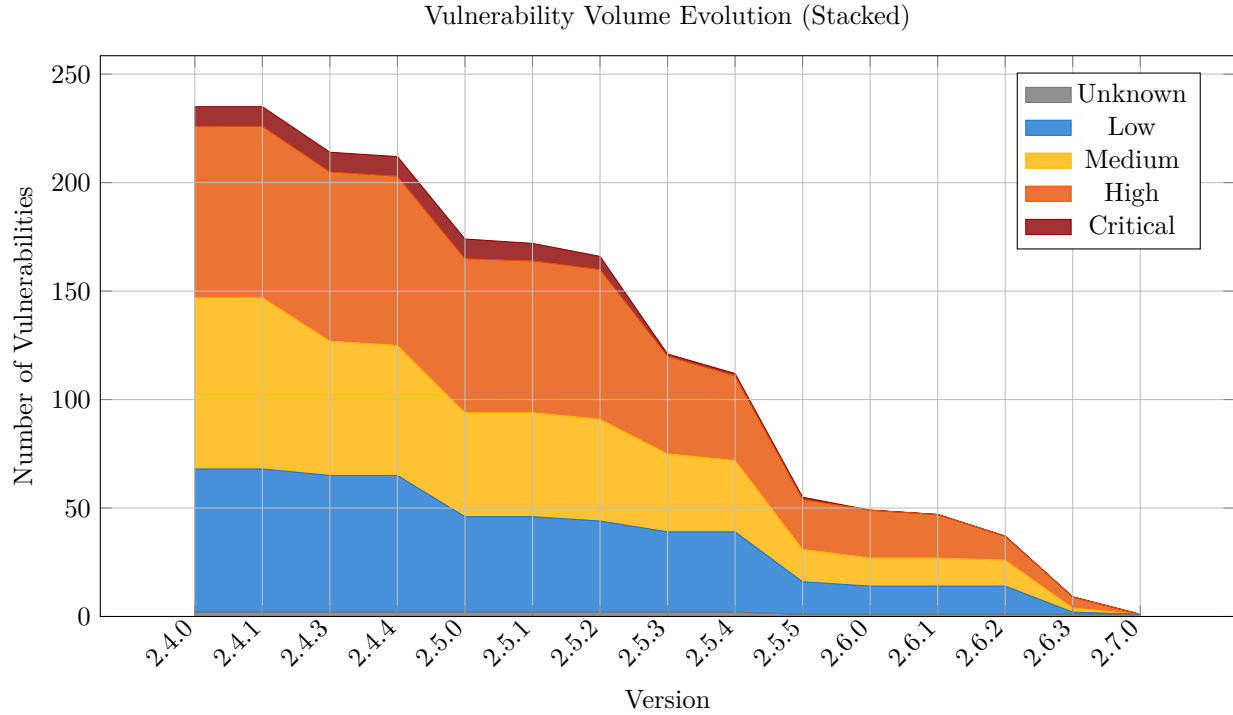


Figure 2: Stacked view showing total vulnerability volume and severity composition

6.4 Version-to-Version Improvement Rate

The following table quantifies the percentage reduction in total vulnerabilities between consecutive releases, highlighting the most impactful updates.

Table 5: Vulnerability Reduction Rate Between Versions

Version Transition	Before	After	Reduction
2.4.0 → 2.4.1	235	235	0.0%
2.4.1 → 2.4.3	235	214	8.9%
2.4.3 → 2.4.4	214	212	0.9%
2.4.4 → 2.5.0	212	174	17.9%
2.5.0 → 2.5.1	174	172	1.1%
2.5.1 → 2.5.2	172	166	3.5%
2.5.2 → 2.5.3	166	121	27.1%
2.5.3 → 2.5.4	121	112	7.4%
2.5.4 → 2.5.5	112	55	50.9%
2.5.5 → 2.6.0	55	49	10.9%
2.6.0 → 2.6.1	49	47	4.1%
2.6.1 → 2.6.2	47	37	21.3%
2.6.2 → 2.6.3	37	9	75.7%
2.6.3 → 2.7.0	9	1	88.9%

The data reveals five major security milestones:

- **v2.5.3:** 27.1% reduction - Alpine 3.23.4 rebase plus a large application-layer cut (Critical drops from 6 to 1)
- **v2.5.5:** 50.9% reduction - Alpine upgrade to 3.24.1 more than halves the OS layer (50 → 20)

- **v2.6.2:** 21.3% reduction - Application-layer hardening on the same Alpine 3.24.1 base (27 → 17 Go findings), notably halving the High count from 20 to 11
- **v2.6.3:** 75.7% reduction - The Alpine 3.24.2 rebase **eliminates all remaining OS-layer vulnerabilities** (20 → 0), clearing the OpenSSL libcrypto3/libssl3 cluster in a single step
- **v2.7.0:** 88.9% reduction - The minor release closes every outstanding OpenBao core advisory, leaving a single unfixable Unknown-severity item and a perfect 100.0/100 weighted score

7 Deep Security Insights

This section provides a more granular analysis of the vulnerability data, distinguishing between different layers of the container image and identifying persistent risks.

7.1 OS vs. Application Vulnerabilities

Distinguishing between vulnerabilities in the base operating system (Alpine Linux) and the application layer (Go binaries and dependencies) is crucial for identifying the primary source of risk.

Table 6: Vulnerability Distribution: OS Layer vs. Application Layer

Version	OS (Alpine)	Application (Go)	Total
2.4.0	111	124	235
2.4.1	111	124	235
2.4.3	105	109	214
2.4.4	105	107	212
2.5.0	75	99	174
2.5.1	75	97	172
2.5.2	75	91	166
2.5.3	50	71	121
2.5.4	50	62	112
2.5.5	20	35	55
2.6.0	20	29	49
2.6.1	20	27	47
2.6.2	20	17	37
2.6.3	0	9	9
2.7.0	0	1	1

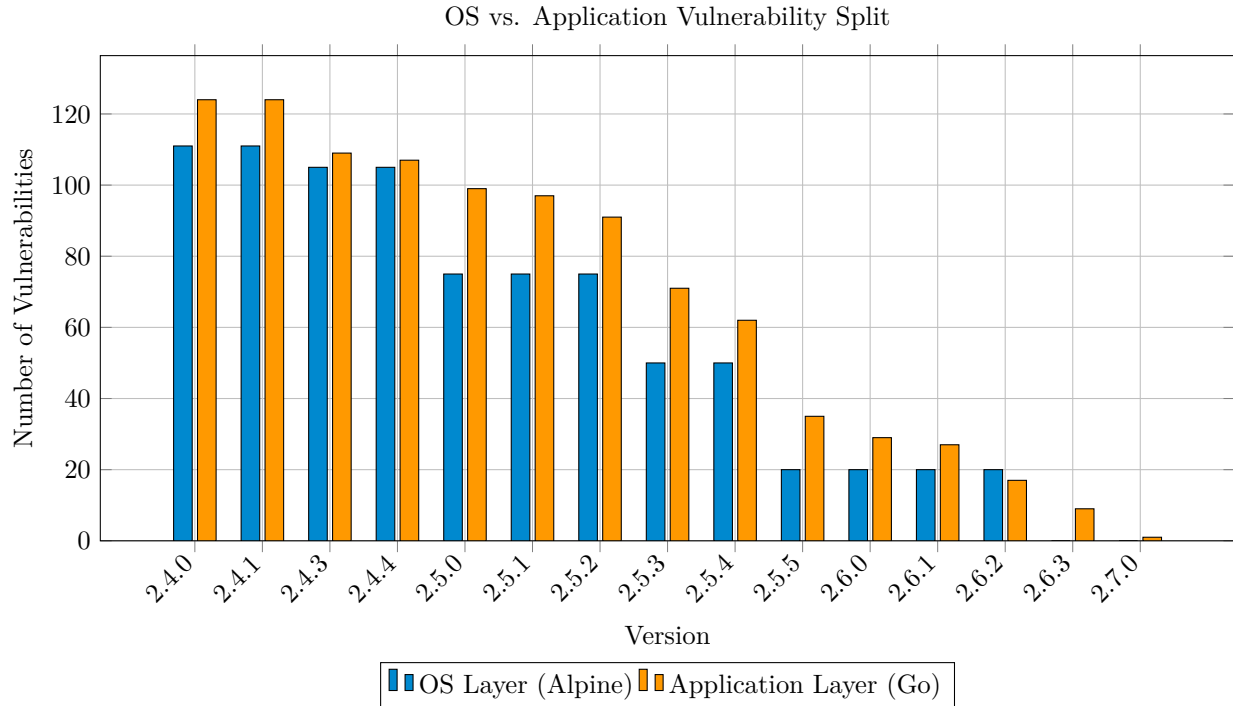


Figure 3: Visualizing the migration of risk from OS to Application layer

The data highlights a decisive milestone in version 2.6.3, where all remaining operating-system vulnerabilities were eliminated (down from 20 in v2.6.2) by the Alpine 3.24.2 rebase — which cleared an entire cluster of OpenSSL `libcrypto3/libssl3` advisories (CVE-2026-14456, CVE-2026-18798, CVE-2026-63072–63076, CVE-2026-75803 and others). The OS layer declines steadily across the series: 111 (v2.4.0) → 75 (v2.5.0–v2.5.2) → 50 (v2.5.3/v2.5.4) → 20 (v2.5.5–v2.6.2) → 0 (v2.6.3 onward). The application layer follows its own downward path, from 124 findings in v2.4.0 to 9 in v2.6.3, and finally to a single item in v2.7.0 once the OpenBao core advisories are resolved. With both layers now effectively clear, the residual surface is no longer a matter of patching but of dependency hygiene.

7.2 Remediation Status (Fixable Vulnerabilities)

A “Fixable” vulnerability is one for which a patched version of the affected package is already available. Analyzing this metric helps measure the potential for immediate risk reduction.

Table 7: Fixable vs. Total Vulnerabilities

Version	Fixable	Total	% Fixable
2.4.0	231	235	98%
2.4.1	231	235	98%
2.4.3	210	214	98%
2.4.4	208	212	98%
2.5.0	173	174	99%
2.5.1	171	172	99%
2.5.2	165	166	99%
2.5.3	120	121	99%
2.5.4	111	112	99%
2.5.5	54	55	98%
2.6.0	48	49	98%
2.6.1	46	47	98%
2.6.2	36	37	97%
2.6.3	8	9	89%
2.7.0	0	1	0%

Across v2.4.0–v2.6.2, 97–99% of identified vulnerabilities have an available fix, which is precisely why upgrading is so effective: almost every finding in an older image is one the maintainers have already addressed downstream. The ratio then falls sharply — 89% in v2.6.3 and 0% in v2.7.0 — but this inversion is a sign of health rather than risk. As the fixable backlog is cleared, the only finding left standing is GO-2026-5932, an advisory against the unmaintained `golang.org/x/crypto/openpgp` package for which no upstream fixed version exists. In other words, v2.7.0’s 0% fixable rate describes a single irreducible item, not an unpatched backlog.

7.3 Top Recurring CVEs

Identifying vulnerabilities that persist across multiple releases helps highlight long-standing security challenges or dependencies.

Table 8: Most persistent CVEs/advisories across the fifteen analyzed versions

CVE ID	Persistence (Releases)
GO-2026-5932	15 (2.4.0 to 2.7.0)
CVE-2024-8185	14 (2.4.0 to 2.6.3)
CVE-2024-9180	14 (2.4.0 to 2.6.3)
CVE-2025-59043	14 (2.4.0 to 2.6.3)
CVE-2025-64761	14 (2.4.0 to 2.6.3)
CVE-2026-45808	14 (2.4.0 to 2.6.3)

In this homogeneous rescan, exactly one item reaches maximum persistence: **GO-2026-5932**, present in all fifteen releases including v2.7.0. It is not a conventional CVE but a Go advisory flagging `golang.org/x/crypto/openpgp` as unmaintained and unsafe by design; because the package is deprecated rather than broken in a patchable way, no fixed version exists and no upgrade can clear it.

The next tier — CVE-2024-8185, CVE-2024-9180, CVE-2025-59043, CVE-2025-64761 and CVE-2026-45808, all in the OpenBao binary itself — stops at 14/15. Each of these was present in every release from v2.4.0 through v2.6.3 and is **resolved in v2.7.0**, together with the two MEDIUM core advisories CVE-2026-46358 and CVE-2026-46405. This is the single most consequential change in the dataset: the v2.7.0 minor release retires the entire backlog of OpenBao application CVEs at once, which is exactly the outcome the project’s versioning policy promises for minor upgrades.

7.4 Detailed Analysis of Top Persistent CVEs

Understanding the nature and impact of the most persistent vulnerabilities provides insight into long-term security challenges.

Table 9: The five core CVEs cleared by v2.7.0, and the one advisory that remains

CVE ID	CVSS	Severity	Package	Description
CVE-2024-8185	7.5	HIGH	openbao (vault core)	Denial of Service when processing Raft Join requests
CVE-2025-59043	7.5	HIGH	openbao (vault core)	Denial of Service via malicious unauthenticated JSON requests
CVE-2024-9180	7.2	HIGH	openbao (vault core)	Operators in the root namespace may elevate their privileges
CVE-2025-64761	7.2	HIGH	openbao (vault core)	Privileged operator identity-group root escalation
CVE-2026-45808	N/A	HIGH	openbao (vault core)	Cross-namespace lease revocation via legacy sys/revoke path bypasses ACL
GO-2026-5932	N/A	UNKNOWN	golang.org/x/crypto	openpgp package unmaintained and unsafe by design — no fix available

The first five entries resided in the OpenBao application binary itself, so — unlike the OS-layer CVEs (musl, OpenSSL, libcap) cleared by the Alpine 3.24.2 rebase in v2.6.3 — they could only be resolved upstream, in code. They were, in **v2.7.0**, which is what closes the gap between a well-maintained base image and a well-maintained application.

That leaves GO-2026-5932 as the sole finding in v2.7.0. It is qualitatively different from the rest: an advisory that the `golang.org/x/crypto/openpgp` package is unmaintained, carrying no CVSS score, no severity rating Trivy can map, and no fixed version. It cannot be patched away — only designed away, by migrating off the deprecated package. Organizations that do not exercise OpenPGP code paths can reasonably treat it as informational.

7.5 Release Timeline and Update Velocity

Understanding the release cadence helps assess the project’s responsiveness to security issues.

Table 10: OpenBao Release Timeline (2025-2026)

Version	Release Date	Days Since Previous	Total Vulnerabilities
2.4.0	2025-08-28	—	235
2.4.1	2025-09-11	14	235
2.4.3	2025-10-22	41	214
2.4.4	2025-11-24	33	212
2.5.0	2026-02-04	72	174
2.5.1	2026-02-23	19	172
2.5.2	2026-03-25	30	166
2.5.3	2026-04-20	26	121
2.5.4	2026-05-20	30	112
2.5.5	2026-06-17	28	55
2.6.0	2026-07-14	27	49
2.6.1	2026-07-22	8	47
2.6.2	2026-08-18	27	37
2.6.3	2026-09-23	36	9
2.7.0	2026-09-23	0	1

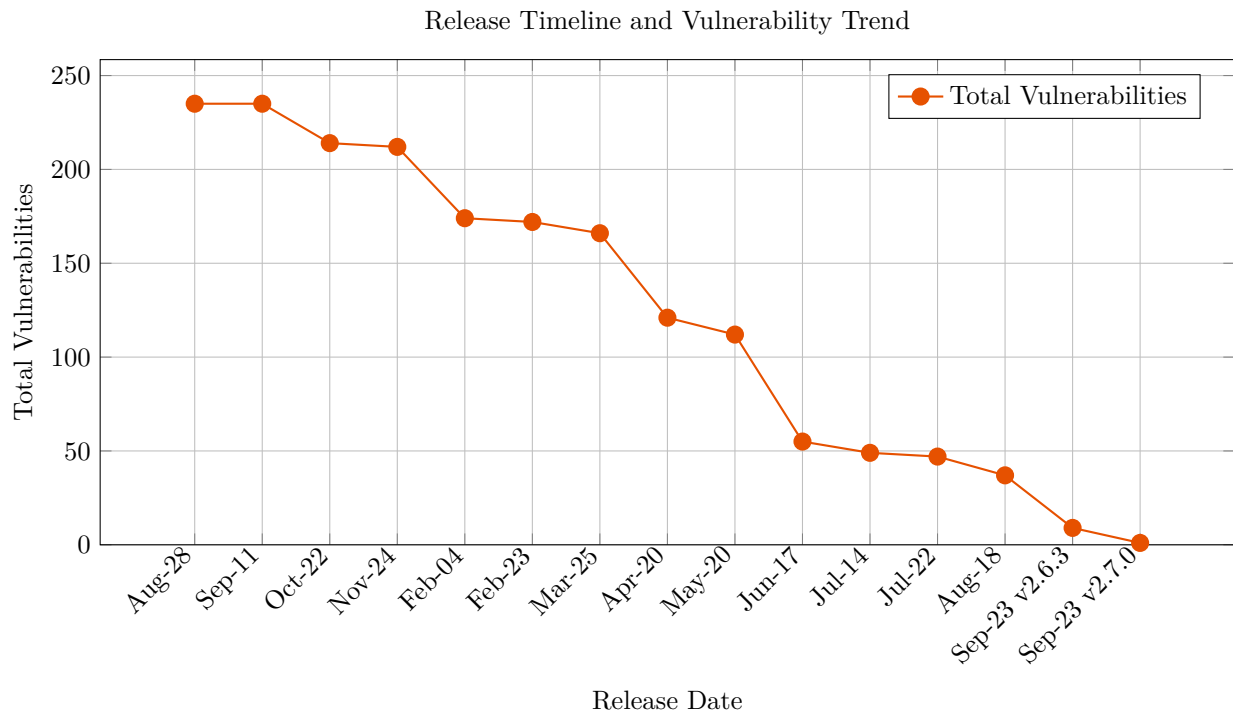


Figure 4: Release frequency and vulnerability reduction over time

The timeline reveals a consistent release cadence averaging ~ 28 days across the 2.5.x and 2.6.x series, punctuated by faster turnarounds when warranted: v2.6.1 shipped just 8 days after v2.6.0. The most striking entry is the tail: **v2.6.3 and v2.7.0 were published on the same day** (2026-09-23, roughly half an hour apart). This is a deliberate pattern rather than an anomaly — v2.6.3 is the final patch of the 2.6 line, carrying the Alpine 3.24.2 rebase to users who must stay on 2.6.x, while v2.7.0 opens the new minor line with the full set of application fixes. Users on either track receive the OS-layer cleanup; only those who move to 2.7.0 also clear the OpenBao core advisories.

8 Measuring CVE Database Drift

Every edition of this report has warned that vulnerability counts move over time as new CVEs are disclosed. This edition can do better than warn: because all scan results are committed to version control, **git history holds the same container images measured at several points in time**. Comparing an image against itself across scan dates isolates database drift exactly, since the image is byte-identical and any movement must come from the database alone.

The scan date below is read from the **CreatedAt** field inside each committed scan file rather than from the commit date, so results are attributed to the day they were produced. The analysis is reproducible with `./drift_analysis.sh`.

Table 11: The same image, measured repeatedly: apparent vulnerability count by scan date

Image	Scan Date	Trivy	Total	Crit	High	Med	Low
v2.4.0	2026-05-21	0.70.0	125	9	45	58	13
v2.4.0	2026-06-22	0.71.2	165	7	49	57	52
v2.4.0	2026-07-16	0.72.0	195	8	63	68	52
v2.4.0	2026-07-23	0.72.0	196	8	64	68	52
v2.4.0	2026-09-24	0.74.0	235	9	79	79	66
v2.5.5	2026-06-18	0.71.1	9	0	5	3	1
v2.5.5	2026-06-22	0.71.2	9	0	5	3	1
v2.5.5	2026-07-16	0.72.0	14	0	6	4	1
v2.5.5	2026-07-23	0.72.0	15	0	7	4	1
v2.5.5	2026-09-24	0.74.0	55	1	23	15	15
v2.6.1	2026-07-23	0.72.0	10	0	6	3	0
v2.6.1	2026-09-24	0.74.0	47	0	20	13	13

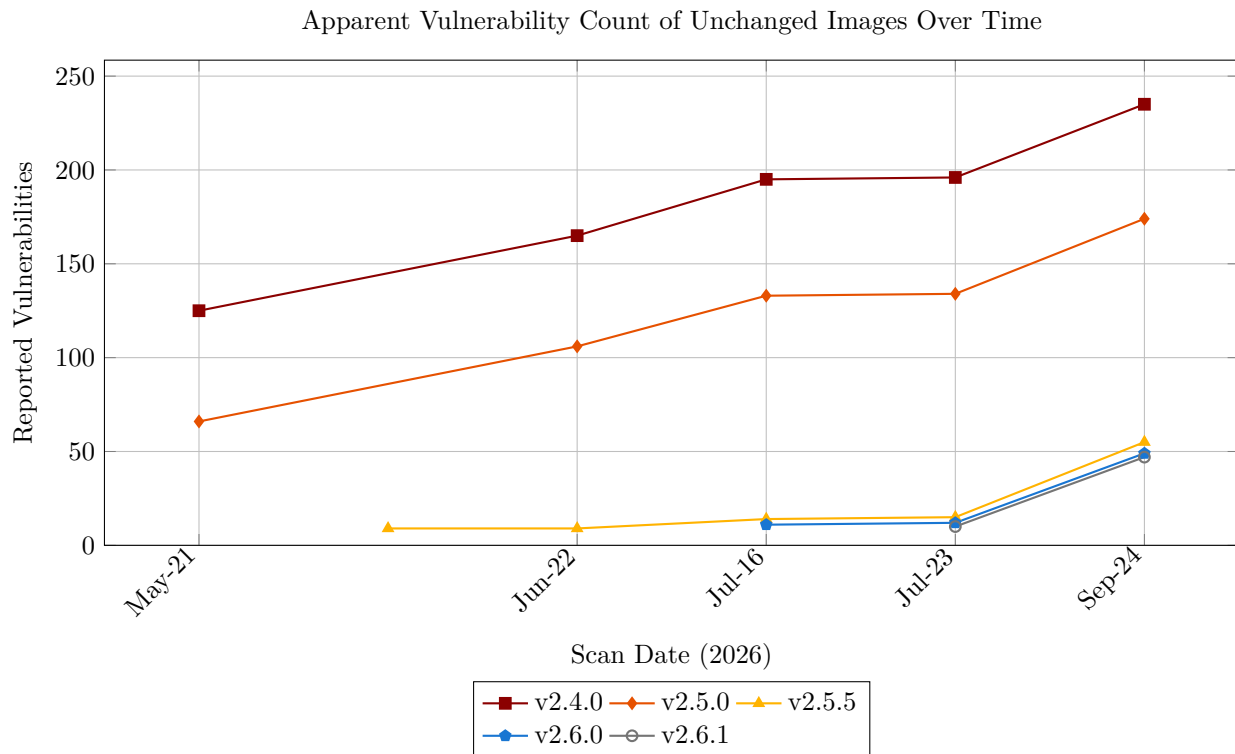


Figure 5: Every line is a *fixed, unchanged* image. All upward movement is database drift.

Not one of these images was rebuilt, yet every line rises. Table 12 quantifies the effect.

Table 12: Drift magnitude per image over its observed window

Image	Window (days)	First	Latest	Change	Per 30 days
v2.4.0	126	125	235	+110 (+88%)	+26.2
v2.5.0	126	66	174	+108 (+164%)	+25.7
v2.5.5	98	9	55	+46 (+511%)	+14.1
v2.6.0	70	11	49	+38 (+345%)	+16.3
v2.6.1	63	10	47	+37 (+370%)	+17.6

An unchanged OpenBao image accumulates roughly **14–26 newly-disclosed findings per month**. Older images drift faster in absolute terms — they carry more packages with more surface — while newer, leaner images drift far more dramatically in *relative* terms, because they start from a small base. v2.5.5 was published at 9 findings and stands at 55 today, a 6-fold increase with no change whatsoever to the artifact.

8.1 Decomposing an Apparent Improvement

This measurement makes it possible to separate real security work from measurement artifact. Consider a reader comparing the previous edition of this report with the current one: the previous edition’s best version, v2.6.1, was published at 10 findings; this edition’s best, v2.7.0, stands at 1. The apparent gain is a modest 9 findings.

That figure is badly wrong in both directions at once:

Component	Findings	What it measures
Naive cross-edition delta	−9	v2.6.1 as published vs. v2.7.0 now
Real improvement	−46	v2.6.1 vs. v2.7.0, <i>both</i> scanned 2026-09-24
Database drift	+37	v2.6.1 vs. itself, 2026-07-23 → 2026-09-24
Sum	−9	reconciles to the naive figure

The engineering work behind v2.7.0 removed **46** findings, five times what the naive comparison suggests. It looked like 9 only because 37 newly-disclosed CVEs landed against the baseline in the same period. A reader comparing published figures across editions would have badly underestimated the release — and, had the direction of work been reversed, drift could equally have disguised a genuine regression as progress.

8.2 Consequences for Reading This Report

Three practical rules follow:

1. **Comparisons within this report are valid.** All fifteen tags were scanned in a single session against one database, so version-to-version differences reflect the images alone.
2. **Comparisons against externally published counts are not valid**, including against earlier editions of this report, unless the scan date and database version match.
3. **An absolute vulnerability count is meaningless without its measurement date.** “v2.5.5 has 9 vulnerabilities” was true on 2026-06-18 and is wrong today by a factor of six, with no dishonesty and no change to the image.

This is also the operational argument for upgrading on a schedule rather than on evidence: a deployed image does not become more secure by standing still, but the measured risk attached to it grows steadily. The drift rates above suggest an unchanged OpenBao deployment accrues on the order of 15–25 newly-disclosed findings per month of inaction.

9 Glossary of Security Terms

CVE (Common Vulnerabilities and Exposures): A list of publicly disclosed computer security flaws. Each entry (e.g., CVE-2024-1234) represents a specific vulnerability in a software package.

CVSS (Common Vulnerability Scoring System): A free and open industry standard for assessing the severity of computer system security vulnerabilities. Scores range from 0 to 10.

Severity Levels:

- **Critical:** Vulnerabilities that could be easily exploited by a remote attacker to gain full control of the system.

- **High:** Vulnerabilities that are difficult to exploit but could result in significant elevated privileges or data loss.
- **Medium:** Vulnerabilities that require specific configurations or local access to exploit.
- **Low:** Vulnerabilities that have very little impact on the system's security.

OCI Image (Open Container Initiative): A standard for container images that ensures portability across different container engines (like Docker or Podman).

Trivy: An open-source vulnerability scanner specifically designed for containers and other artifacts.

10 Conclusion

The analysis indicates that OpenBao has undergone sustained and substantial security hardening. Moving from version 2.4.0 to the latest 2.7.0 release reduces the vulnerability count from 235 to a single finding — a **99.6% reduction** — and eliminates every Critical, High, Medium and Low-severity item, yielding a perfect 100.0/100 weighted Security Posture Score.

Two distinct forces produced that result, and the data separates them cleanly. The **operating-system layer** was cleared by disciplined base-image maintenance: all fifteen analyzed releases shipped the newest Alpine patch available on their build date, and the Alpine 3.24.2 rebase in v2.6.3 removed the last 20 OS findings in one step. The **application layer** was cleared by upstream code fixes: v2.7.0 retires the seven OpenBao core advisories — five HIGH, two MEDIUM — that had persisted in all fourteen prior releases. Neither improvement alone would have been sufficient; together they leave only GO-2026-5932, an advisory on the deprecated `golang.org/x/crypto/openpgp` package that carries no fix and no severity score.

OpenBao's adherence to Semantic Versioning and its low-risk minor release strategy — similar to PostgreSQL's versioning philosophy — ensures that upgrading to the latest minor version is not only safe but actively recommended. The v2.6.3/v2.7.0 pairing illustrates the point precisely: both shipped on 2026-09-23, with v2.6.3 delivering the base-image cleanup to users pinned to the 2.6 line and v2.7.0 delivering both that cleanup and the application fixes. The project supports conservative upgraders without holding back those who move forward.

One methodological point deserves emphasis, and this edition measures it rather than merely asserting it (Section 8). Every version's counts here are **higher** than in the previous edition — v2.6.1 moved from 10 findings to 47, v2.5.5 from 15 to 55 — purely because this rescan used a Trivy database four months newer. No image changed; the set of known CVEs did. Because the scan results are committed to version control, the same images can be re-read as they were measured on five separate dates, which shows an unchanged OpenBao image accruing roughly 14–26 newly-disclosed findings per month.

That measurement materially changes how this release should be read. Comparing the previous edition's best version (v2.6.1 at 10) against this edition's best (v2.7.0 at 1) suggests a 9-finding improvement. Scanning both on the same day shows the real figure is **46** — five times larger — with 37 findings of database drift masking it. Relative comparisons within this report remain valid; comparisons against externally published figures, earlier editions of this report included, do not.

Organizations running older minor versions should prioritize upgrading to **v2.6.3 or later**, and to v2.7.0 where the minor bump is acceptable. This rescan places v2.5.2 through v2.5.4 in the MEDIUM risk band and everything earlier in the HIGH band, while v2.5.5 onward sits in LOW. As demonstrated throughout, staying on an outdated minor version exposes systems to substantially higher risk than the minimal effort a minor upgrade requires.